

Universidade de São Paulo



Java Library

SCC0504

Programação Orientada a Objetos

Alunos

Felipe Galvão Prazeres - n^o 16828948

Laura Nordi Zambom - n^o 14655491

São Carlos

2026

1 Introdução

O objetivo principal desse trabalho é simular o gerenciamento eficiente de uma biblioteca, permitindo o controle de livros do acervo, o cadastro de usuários e o registro de empréstimos e devoluções. Para isso, o sistema foi inteiramente desenvolvido em Java, utilizando o paradigma de Orientação a Objetos e a biblioteca **Java Swing** para a construção de uma Interface Gráfica de Usuário (GUI) interativa e dividida em abas.

Além das operações fundamentais de um sistema CRUD (*Create, Read, Update, Delete*) e da persistência local de dados através de arquivos binários, o projeto foi expandido para englobar as funcionalidades opcionais. Entre essas adições, destacam-se o controle de acesso baseado em papéis (Administrador vs. Bibliotecário), o cálculo automático de multas por atraso e a visualização do histórico de empréstimos.

2 Requisitos do Sistema

De acordo com a descrição da atividade "JavaLibrary - Library Management System", todos os requisitos exigidos foram desenvolvidos, além da implementação das funcionalidades adicionais, como descrito abaixo.

2.1 Principais Requisitos Implementados:

- **Gestão de Livros:** O sistema permite adicionar, editar, remover e buscar livros no acervo (por título, autor ou ISBN). Para mais, controla a quantidade de cópias disponíveis de cada obra.
- **Gestão de Usuários (Patrons):** O sistema permite cadastro, edição, remoção e busca de usuários utilizando parâmetros como nome, identificador (ID) e informações de contato.
- **Empréstimos (Loans):** O sistema permite realizar empréstimos e devoluções, garantindo a integridade dos dados, impedindo o empréstimo de livros sem cópias disponíveis e bloqueando a exclusão de usuários que possuam devoluções pendentes.
- **Interface Gráfica (GUI):** Utilizando a biblioteca **Java Swing**, a aplicação apresenta uma interface dividida em abas distintas para Livros, Usuários e Empréstimos, facilitando a navegação.
- **Persistência de Dados:** O estado do sistema é salvo localmente em disco na pasta **data/** através de identificação (com o uso de um Serial Number) de objetos (arquivos **.dat**), garantindo que os dados não sejam perdidos ao fechar o programa.
- **Tratamento de Exceções:** Utilização de blocos **try-catch** e exceções customizadas para capturar erros lógicos e exibir mensagens de aviso ao usuário via interface gráfica.

2.2 Funcionalidades Extras

- **Controle de Acesso por Papéis (RBAC):** Tela de Login com diferenciação de permissões. Usuários com perfil de "Administrador" possuem acesso total, enquanto "Bibliotecários" possuem permissões restritas a adição e edição.
- **Cálculo de Multas por Atraso:** O sistema calcula multas automaticamente para devoluções realizadas após o prazo estabelecido (taxa de \$1.50 por dia de atraso).
- **Histórico de Empréstimos:** Visualização do histórico de locações por usuário, incluindo status ativo/devolvido e multas pagas.

3 Descrição do Projeto e Implementação

O projeto foi estruturado adotando o padrão de arquitetura MVC (Model-View-Controller), garantindo a separação clara de responsabilidades e facilitando a manutenção do código. A divisão estrutural foi definida da seguinte forma:

- **Modelos (Models):** Book, Patron, Loan, e User.
- **Visões (Views - GUI):** LoginFrame, MainFrame, BookPanel, PatronPanel, LoanPanel.
- **Controladores (Services/Managers):** AuthManager e LibraryManager.

3.1 Gestão de Entidades e Regras de Negócio

As entidades principais do domínio (Book, Patron, Loan e User) formam a camada *Model*. Elas são estritamente encapsuladas, protegendo seus atributos via modificadores `private` e liberando o acesso apenas por `getters` e `setters`.

A camada *Controller*, por sua vez, é centralizada nas classes `LibraryManager` e `AuthManager`. O `LibraryManager` gerencia as listas de dados principais e aplica as regras de negócio. Para implementar as funcionalidades de busca e validação (como impedir a exclusão de um usuário com pendências), optou-se por utilizar a **Streams API** do Java (métodos `filter()` e `anyMatch()`), substituindo laços de repetição tradicionais por operações mais funcionais e limpas.

3.2 Interface Gráfica e Compartilhamento de Estado

A interface (camada *View*) foi construída com Java Swing, utilizando um `JFrame` principal contendo um `JTabbedPane` para dividir as seções (`BookPanel`, `PatronPanel`, `LoanPanel`).

É importante ressaltar o uso de **Injeção de Dependências** via construtor: o `LibraryManager` e o `AuthManager` são instanciados uma única vez na classe `Main` e suas referências de memória são repassadas para os painéis. Isso garante que todas as abas operem sobre o mesmo estado de dados em memória simultaneamente, evitando inconsistências.

3.3 Persistência com Generics

Para o requisito de persistência de dados em arquivos binários `.dat`, escolheu-se utilizar a interface nativa `Serializable` em todas as classes de modelo.

As operações de I/O (Input/Output) foram isoladas na classe utilitária estática `FileManager`. Para evitar duplicação de código ao ler as diferentes listas de objetos usando `ObjectInputStream`, implementamos o método `loadFromFile` utilizando **Generics** (`<T>`). Dessa forma, o mesmo método é capaz de realizar o *casting* dinâmico e seguro para listas de Livros, Usuários ou Empréstimos.

3.4 Tratamento de Exceções via Herança

Para reportar falhas de regras de negócio (como login inválido ou falta de cópias), utilizou-se herança para criar classes de exceção customizadas, tais como `BookAlreadyOnLoanException`, `PatronHasActiveLoansException`, `AuthenticationException`, que estendem a classe mãe `Exception`.

Isso permite que a camada *Controller* lance os erros de forma semântica, deixando para a *View* capturá-los em blocos `try-catch` e exibir, na tela, a mensagem correta ao usuário, através do `JOptionPane.showMessageDialog()`.

3.5 Cálculo de Multas

Para a funcionalidade extra de controle de atrasos e multas, implementou-se a lógica utilizando a API `java.time.LocalDate`. O cálculo financeiro é realizado com `ChronoUnit.DAYS.between()`, que calcula a diferença, em dias, entre a data de vencimento estimada na hora do empréstimo e a data de devolução efetiva, multiplicando o resultado pela constante de taxa diária.

3.6 Princípios de POO Aplicados

- **Encapsulamento:** Todas as classes de modelo protegem seus estados internos utilizando atributos `private` e expõem modificações apenas através de `getters` e `setters` controlados. O `LibraryManager` encapsula as estruturas de dados principais (`List<Book>`, `List<Patron>`, etc.), impedindo que a GUI manipule as listas diretamente. As ações ocorrem através de métodos como `checkoutBook()`, protegendo as regras de negócio.
- **Herança:** A herança de classes foi aplicada no tratamento de erros do sistema, com a criação de exceções semânticas personalizadas (`AuthenticationException`, `BookAlreadyOnLoanException` e `PatronHasActiveLoansException`) que herdam da classe nativa `Exception` do Java. Isso permite que a camada de controle "jogue"(`throw`) o erro, e a camada visual o capture (`catch`) para exibir um `JOptionPane` com a devida mensagem ao usuário.
- **Abstração:** O sistema oculta a complexidade das operações internas. Por exemplo, a classe estática `FileManager` abstrai toda a lógica de baixo nível de entrada e saída (I/O) de arquivos binários e fluxos de dados. Da mesma forma, a interface gráfica (*View*) apenas invoca métodos de alto nível como `checkoutBook()` no `LibraryManager`, sem saber os detalhes de atualizações de listas, instâncias de datas ou geração de UUIDs.

- **Polimorfismo:** Aplica-se polimorfismo em todas as classes de modelo (Book, Patron, Loan, User) ao fazer a reescrita dos métodos herdados da classe supermãe `Object`, especificamente `equals()`, `hashCode()` e `toString()`. O uso do `equals()` customizado para comparar objetos pelos seus identificadores únicos foi crucial para o funcionamento ideal de métodos como `indexOf()` e `remove()` do `LibraryManager`.

4 Comentários Sobre o Código

Para facilitar a leitura, compreensão e manutenção do código, foram incluídos comentários diretamente nos arquivos `.java`, documentando o objetivo dos métodos e das regras de negócio. Dentre as implementações técnicas, destacam-se duas que exigem maior compreensão da linguagem, explicadas e comentadas abaixo.

4.1 Type Erasure e Supressão de Avisos em Generics

Na classe `FileManager`, foi criado o método universal `loadFromFile` utilizando Generics (`<T>`). Devido a um mecanismo do Java chamado *Type Erasure*, que faz o apagamento de tipos em tempo de execução, o compilador não consegue garantir 100% de segurança ao fazer o *cast* de um objeto genérico vindo do arquivo para uma `List<T>`.

Isso gera um *warning* (aviso de compilação). Como a arquitetura usada garante que apenas as listas corretas sejam gravadas e lidas em seus respectivos arquivos, o aviso é um falso positivo. Para lidar com isso, utilizamos a anotação `@SuppressWarnings("unchecked")`:

Trecho de `FileManager.java`

```

1 // a arquitetura ja garante a consistencia no arquivo lido
2 // a anotacao @SuppressWarnings("unchecked") silencia o
  aviso do compilador gerado
3 // pelo "Type Erasure"
4 @SuppressWarnings("unchecked")
5 private static <T> List<T> loadFromFile(String filePath) {
6     File file = new File(filePath);
7
8     // retorna uma lista vazia caso seja a primeira execucao
      do sistema
9     if (!file.exists()) {
10         return new ArrayList<>();
11     }
12
13     // o uso de try-with-resources garante que o
      ObjectInputStream sera fechado automaticamente
14     // ao final do bloco, prevenindo vazamentos de memoria
15     try (ObjectInputStream ois = new ObjectInputStream(new
      FileInputStream(file))) {
16
17         // Faz o cast for ado para a lista generica, que
      motivou a anotacao acima
18         return (List<T>) ois.readObject();

```

```

19
20     } catch (IOException | ClassNotFoundException e) {
21         System.err.println("Error loading from file: " + e.
22             getMessage());
23         return new ArrayList<>();
24     }

```

4.2 Comportamento da Busca e Remoção de Objetos

Outro detalhe crucial para o funcionamento do `LibraryManager` foi a necessidade de sobrescrever os métodos `equals()` e `hashCode()` nas classes de modelo. Nativamente, o Java compara se dois objetos ocupam o mesmo espaço de memória.

No caso da entidade `Book`, dois livros são idênticos se possuírem o mesmo ISBN. Essa sobrescrita foi essencial para que os métodos internos da biblioteca `java.util.List`, como `books.indexOf(oldBook)` e `books.remove(book)`, funcionassem de forma lógica, e não referencial.

Trecho de `Book.java`

```

1 // sobrescreve o metodo equals da superclasse nativa Object.
2 // por padrao, o Java compara apenas os enderecos de memoria
3 // (referencias).
4 // entao, implementamos a "igualdade semantica": dois
5 // objetos diferentes na memoria
6 // serao considerados o mesmo livro pelo Java se possuirem o
7 // mesmo ISBN.
8 @Override
9 public boolean equals(Object o) {
10     // se as variaveis apontam para o mesmo endereco, ja
11     // retorna true
12     if (this == o) return true;
13
14     // verifica se o objeto passado e nulo ou se pertence a
15     // outra classe
16     if (o == null || getClass() != o.getClass()) return
17         false;
18
19     // faz o cast seguro
20     Book book = (Book) o;
21
22     // compara exclusivamente o identificador unico (ISBN).
23     // uso de Objects.equals() para evitar
24     // NullPointerException caso o ISBN seja null.
25     return Objects.equals(isbn, book.isbn);
26 }
27
28 // sempre que sobrescrever equals(), e obrigatorio
29 // sobrescrever o hashCode()
30 // isso garante que livros iguais gerem o mesmo hash,
31 @Override
32 public int hashCode() {

```

```
25     return Objects.hash(isbn);  
26 }
```

5 Plano de Testes

O plano de validação do sistema foi estruturado em duas abordagens: execução manual de teste de interface (focados na experiência do usuário e regras da GUI) e testes automatizados (focados na integridade das classes controladoras).

5.1 Passo a Passo dos Testes Manuais

1. Teste de Autenticação e Permissões:

- Iniciar o sistema e realizar login com credenciais de Bibliotecário (`librarian` / `lib`).
- Navegar pelas abas e verificar se os botões de exclusão (*Delete*) estão bloqueados ou invisíveis.
- Fazer logout, entrar com credenciais de Administrador (`admin` / `admin`) e verificar se os botões de exclusão estão habilitados.

2. Teste de Integridade Referencial (Deleção Segura):

- Na aba de Usuários (*Patrons*), cadastrar um novo usuário.
- Na aba de Empréstimos (*Loans*), associar um empréstimo ativo a este usuário.
- Retornar à aba de Usuários e tentar deletar o registro. O sistema deve exibir um `JOptionPane` com um aviso de erro e bloquear a exclusão.

3. Teste de Empréstimo e Controle de Estoque:

- Cadastrar um livro com o estoque de exata 1 cópia disponível.
- Realizar o empréstimo deste livro. O sistema deve reduzir a quantidade disponível para 0 na tabela.
- Tentar realizar um segundo empréstimo do mesmo livro. O sistema deve bloquear a operação e informar a falta de estoque.

4. Teste de Cálculo de Multas por Atraso:

- Simular a devolução de um livro cuja data atual seja superior em 5 dias à data de devolução esperada (`dueDate`).
- Verificar no recibo ou na tabela se o sistema gerou automaticamente a multa no valor de \$7.50 (5 dias de atraso × \$1.50).

5. Teste de Persistência em Disco:

- Adicionar um livro, um usuário e criar um empréstimo.
- Fechar o aplicativo de forma abrupta.
- Reabrir a aplicação e verificar se os dados cadastrados foram restaurados perfeitamente a partir dos arquivos `.dat`.

5.2 Plano de Testes Automatizados (JUnit)

A camada lógica central do sistema (*Model* e *Controller*) será validada através de suítes de testes automatizados utilizando o *framework* **JUnit 5**. O objetivo é garantir que os métodos das classes `LibraryManager` e `AuthManager` funcionem e lancem as exceções corretamente, sem depender de cliques na tela.

O trecho de código a seguir demonstra o planejamento dos testes unitários para a classe gerenciadora, simulando falta de estoque e atraso na devolução:

Trecho de `LibraryManagerTest.java`

```
1  @Test
2  public void testCheckoutBookFailsNoCopies() throws
   BookAlreadyOnLoanException {
3      // 1. Preparacao do ambiente do teste
4      Book b = new Book("Clean Code", "Robert C. Martin", "456", 1);
5      Patron p1 = new Patron("P1", "John", "john@email.com");
6      Patron p2 = new Patron("P2", "Jane", "jane@email.com");
7
8      libraryManager.addBook(b);
9      libraryManager.addPatron(p1);
10     libraryManager.addPatron(p2);
11
12     // 2. Acao principal: O primeiro emprestimo consome a
        unica copia
13     libraryManager.checkoutBook(b, p1);
14
15     // 3. Verificacao: O teste so passa se o sistema jogar a
        excecao na segunda tentativa
16     assertThrows(BookAlreadyOnLoanException.class, () -> {
17         libraryManager.checkoutBook(b, p2);
18     });
19 }
20
21 @Test
22 public void testReturnBookWithFine() throws
   BookAlreadyOnLoanException {
23     // 1. Preparacao
24     Book b = new Book("Refactoring", "Martin Fowler", "789",
        1);
25     Patron p = new Patron("P1", "Alice", "alice@email.com");
26     libraryManager.addBook(b);
27     Loan loan = libraryManager.checkoutBook(b, p);
28
29     // 2. Acao principal: Forcando a devolucao com exatos 2
        dias de atraso
30     LocalDate returnDate = loan.getDueDate().plusDays(2);
31     libraryManager.returnBook(loan, returnDate);
32
33     // 3. Verificacao rigorosa dos calculos de multa e
        estoque
34     assertEquals(3.0, loan.getFineAmount()); // Multa
        esperada: 2 * $1.50 = $3.0
35     assertEquals(1, b.getAvailableCopies()); // Estoque:
```


6 Resultados dos Testes

A execução dos roteiros de teste confirmou o funcionamento ideal do sistema de acordo com as especificações exigidas, sem apresentar falhas de execução (*crashes*). Abaixo detalhamos os resultados práticos documentados com as capturas de tela do sistema.

6.1 Resultados da Validação Manual (GUI)

1. Autenticação e Permissões (RBAC)

Aprovado. O login com credenciais de Administrador habilitou todos os recursos de gestão. Em contrapartida, ao acessar o sistema com o perfil de Bibliotecário, botões com funções destrutivas foram corretamente ocultados, validando o controle de acesso restrito.

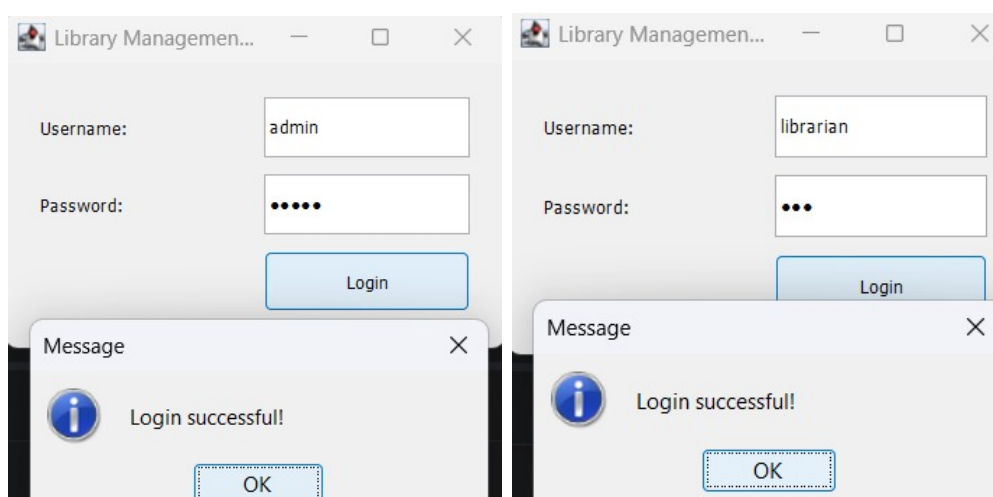


Figura 1: Interfaces de login evidenciando os níveis de permissão: Administrador (esquerda) e Bibliotecário (direita).

2. Gestão de Acervo e Usuários (CRUD e Buscas)

Aprovado. As operações de cadastro e atualização funcionaram sem erros. A pesquisa em tempo real atualizou as tabelas corretamente utilizando diferentes chaves de busca.

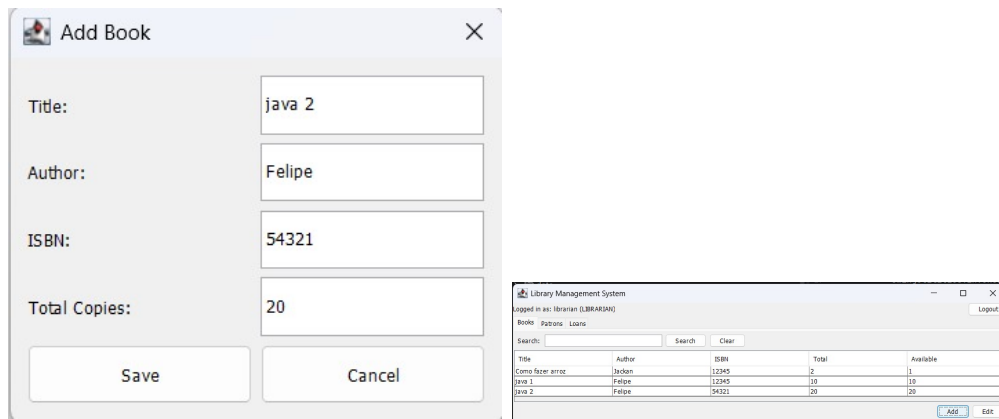


Figura 2: Fluxo de adição de um novo livro ao acervo.

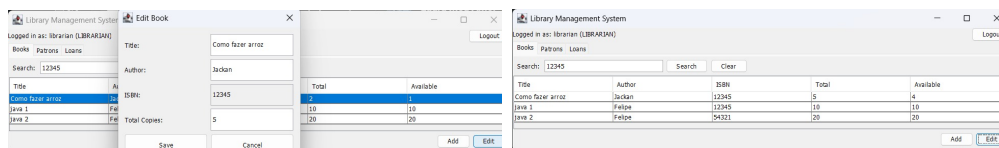


Figura 3: Edição e atualização das informações de um livro existente.

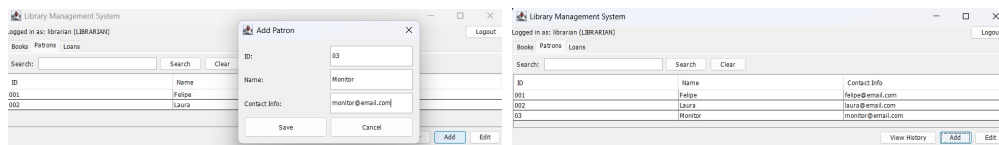


Figura 4: Cadastro de novo usuário (Patron) concluído com sucesso.

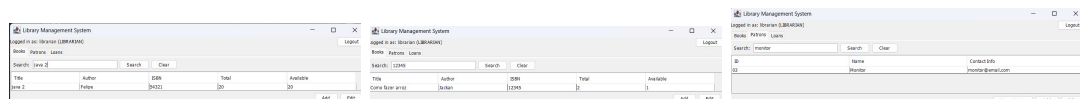


Figura 5: Sistema de busca dinâmica operando com diferentes filtros.

3. Empréstimo e Controle de Estoque

Aprovado. O sistema deduziu a cópia do estoque corretamente. Ao tentar emprestar um livro esgotado, a operação foi interceptada pela exceção dedicada.

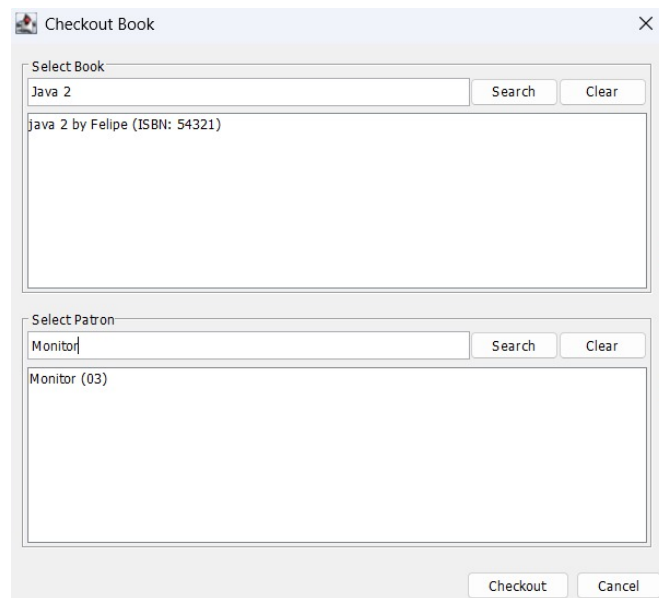


Figura 6: Registro de um novo empréstimo no sistema.

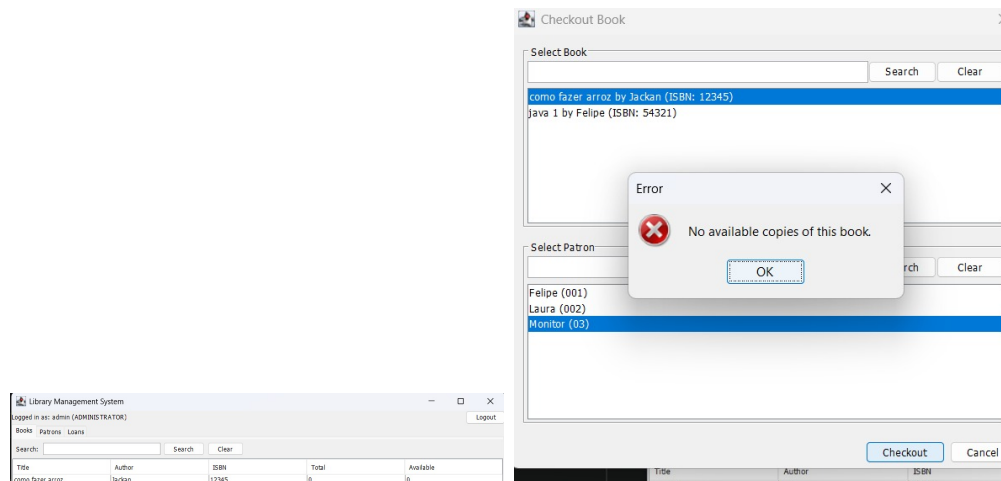


Figura 7: Bloqueio automático de empréstimo por falta de cópias disponíveis.

4. Devoluções, Histórico e Cálculo de Multas

Aprovado. O processamento de devoluções restaurou as cópias disponíveis. Simulações com datas de atraso aplicaram corretamente a penalidade matemática, registrando-a no perfil do usuário.

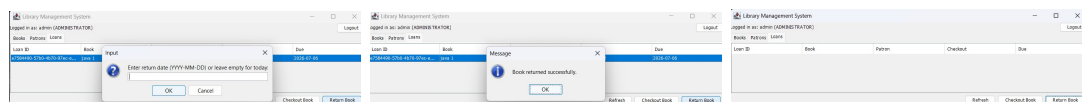


Figura 8: Fluxo de devolução de livro dentro do prazo e atualização automática da interface.

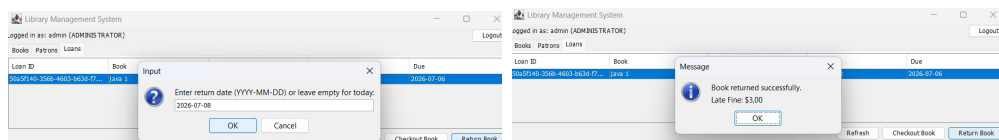


Figura 9: Simulação de devolução com atraso e aplicação automática da taxa financeira.

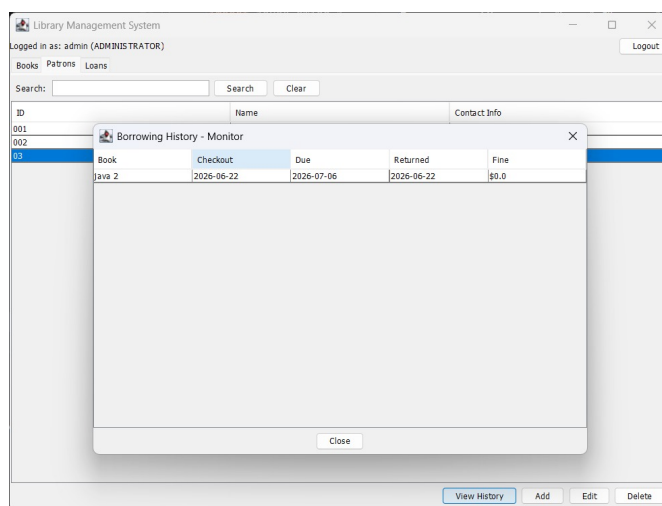


Figura 10: Verificação do registro de pendências e multas no histórico do usuário.

5. Integridade Referencial e Persistência

Aprovado. Exclusões não permitidas foram barradas e a gravação de arquivos .dat conservou os dados.

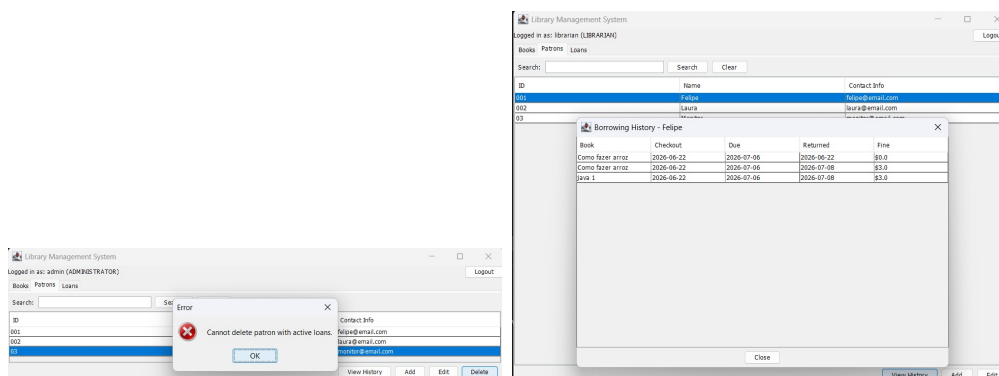


Figura 11: Erro gerado ao tentar deletar usuário com empréstimo ativo (esquerda) e prova de persistência dos arquivos no disco (direita).

6.2 Resultados dos Testes Automatizados (JUnit)

A execução da suíte de testes unitários (`AuthManagerTest` e `LibraryManagerTest`) finalizou com 100% de aprovação (*Green Bar*). O *framework* atestou que as classes de domínio dispararam as exceções customizadas adequadamente e processam os cálculos matemáticos de forma isolada, comprovando a robustez da arquitetura orientada a objetos.

✓	services (com.library)	445 ms
✓	AuthManagerTest	138 ms
✓	testFailedLogin()	124 ms
✓	testSuccessfulLogin()	7 ms
✓	testLogout()	7 ms
✓	LibraryManagerTest	307 ms
✓	testReturnBookWithFin	193 ms
✓	testAddAndSearchBook	41 ms
✓	testCheckoutBookSucc	37 ms
✓	testCheckoutBookFails	36 ms

Figura 12: Resultados da execução dos testes automatizados via JUnit, evidenciando a aprovação em todos os cenários.

7 Procedimentos para Rodar o Código

Este projeto utiliza **Maven** para o gerenciamento de dependências, especificamente para a integração do JUnit 5 nas rotinas de teste. Abaixo estão descritos os requisitos e as instruções para compilar e executar o sistema.

7.1 Pré-requisitos

- **Java Development Kit (JDK):** Versão 8 ou superior (Recomendado: JDK 17).
- **Maven:** Instalado no sistema operacional **OU** execução embarcada via uma IDE compatível (como IntelliJ IDEA ou Eclipse).

7.2 Execução via IntelliJ IDEA (Recomendado)

1. Abra o IntelliJ IDEA.
2. Acesse **File > Open** e selecione o diretório raiz **LibraryApp**.
3. Aguarde a IDE carregar o arquivo **pom.xml** e realizar o download automático das dependências.
4. No explorador de arquivos, navegue até **src/main/java/com/library/Main.java**.
5. Clique no botão verde de execução (*Run*) localizado ao lado da assinatura do método **main**.
6. A tela de autenticação (*Login*) será exibida. Utilize as seguintes credenciais padrão:
 - **Administrador:** Usuário: **admin** | Senha: **admin**
 - **Bibliotecário:** Usuário: **librarian** | Senha: **lib**

7.3 Execução via Linha de Comando

Caso possua o Maven configurado nas variáveis de ambiente do sistema, siga os passos:

1. Abra um terminal e navegue até a pasta raiz `LibraryApp`.
2. Compile o projeto executando o comando:

```
1 mvn compile
```

3. Inicie a aplicação com o comando:

```
1 mvn exec:java -Dexec.mainClass="com.library.Main"
```

7.4 Execução dos Testes Automatizados

Para rodar a suíte de validação do JUnit, escolha uma das abordagens:

- **Via Linha de Comando (Maven):** No terminal, dentro da pasta raiz, execute:

```
1 mvn test
```

- **Via IntelliJ IDEA:**

1. Navegue até a pasta de testes em `src/test/java/com/library/services`.
2. Clique com o botão direito sobre o pacote `com.library.services`.
3. Selecione a opção Run 'Tests in com.library.services'.

8 Problemas e Desafios

Durante o desenvolvimento e validação do sistema, enfrentamos desafios arquiteturais e de usabilidade que exigiram refatoração, solucionados da seguinte forma:

- **Sincronização de Estado na GUI:** Ações em uma aba (ex: realizar empréstimo) não atualizavam os dados nas outras (ex: estoque de livros na tabela principal).

Solução: Para evitar forte acoplamento, centralizamos os dados no `LibraryManager` e adicionamos um `ChangeListener` no `JTabbedPane` do `MainFrame`. Ao trocar de aba, o método `refreshTable()` é acionado, atualizando a tela com os dados mais recentes do backend.

- **Escalabilidade na Interface de Empréstimo:** O uso inicial de `JComboBox` (menus suspensos) para selecionar livros e usuários mostrou-se ineficiente em termos de usabilidade (UX) para grandes volumes de dados.

Solução: Refatoramos o método `showCheckoutDialog()`, substituindo as listas suspensas por componentes `JList` dentro de `JScrollPane`, integrados a campos de texto (`TextField`) com busca dinâmica.

- **Sobrecarga de Eventos na Pesquisa (*Debounce*):** O filtro de texto em tempo real via `DocumentListener` gerava excesso de processamento na thread do Swing (EDT) a cada tecla pressionada.
Solução: Implementamos um mecanismo de *debounce* usando `javax.swing.Timer` (500ms). A busca no banco de dados agora só é executada após o usuário fazer uma breve pausa na digitação, otimizando a responsividade.
- **Filtragem Incompleta via *Streams*:** A busca de usuários inicial estava ignorando as Informações de Contato, que era um parâmetro obrigatório do projeto.
Solução: Expandimos a expressão lambda do filtro da *Streams API* localizada no método `LibraryManager.searchPatrons`, adicionando a condição `p.getContactInfo().toLowerCase().contains(query)`.

9 Conclusão

O sistema foi desenvolvido de forma satisfatória e estável, empregando corretamente as estruturas da linguagem Java e os paradigmas da Orientação a Objetos. Atingimos a totalidade dos requisitos obrigatórios, bem como dos critérios de aprofundamento, resultando em um código escalável e de fácil manutenção.